

Correction détaillée — La boucle « for »

Exercice 1

Que renvoie `somme(4)` ?

```
1 def somme(n):
2     s=0
3     for i in range(n+1):
4         s=s+i
5     return s
```

► **Principe.** `range(n+1)` parcourt $0, 1, 2, \dots, n$. La fonction calcule donc la somme $0 + 1 + 2 + \dots + n$. Pour $n = 4$, `range(5)` donne $0, 1, 2, 3, 4$.

► **Trace pour `somme(4)`.**

i	(départ)	0	1	2	3	4
s	0	0	1	3	6	10

► La fonction renvoie donc **10** (c'est la somme des entiers de 0 à 4, soit $\frac{4 \times 5}{2} = 10$).

Exercice 2

5 000 € placés à 2,5 % par an ; somme disponible au bout de n années.

► **Idée.** Augmenter une somme de 2,5 % revient à la **multiplier par 1,025**. On répète cette opération une fois par année à l'aide de la boucle.

```
1 def compte(n):
2     c = 5000
3     for i in range(n):
4         c = c * 1.025
5     return c
```

► **`range(n)`** fait tourner la boucle n fois : au bout de n années, $c = 5000 \times 1,025^n$. (On pourrait aussi écrire $c = c + c \times 2,5/100$.)

► **Exemples.** `compte(1)` = $5000 \times 1,025 = 5\,125$ € ; `compte(10)` $\approx 6\,400,42$ €.

Exercice 3

Parmi les quatre fonctions *a*, *b*, *c*, *d*, lesquelles affichent 15 ? Qu'affichent les autres ?

- On calcule la somme *S* dans chaque cas en repérant les valeurs prises par *i*.
 - **a.** `range(6)` : $0 + 1 + 2 + 3 + 4 + 5 = 15$. (affiche 15)
 - **b.** `range(1,8,2)` : $1 + 3 + 5 + 7 = 16$. (affiche 16)
 - **c.** `range(1,6)` : $1 + 2 + 3 + 4 + 5 = 15$. (affiche 15)
 - **d.** `range(3,9,2)` : $3 + 5 + 7 = 15$. (affiche 15)
- 1. On entoure donc les fonctions **a**, **c** et **d** (elles affichent 15).
- 2. La seule autre fonction est **b**, qui affiche 16.
- **À retenir.** Tout se joue sur les paramètres de `range(début, fin, pas)` : *fin* est *exclue*, et le *pas* permet de ne prendre qu'un nombre sur deux.

Exercice 4

Fonction *de* qui compte les 6 obtenus.

```

1 from random import randint
2 def de(n):
3     nbre_six=0
4     for i in range(n):
5         if randint(1,6)==6:
6             nbre_six=nbre_six+1
7     return nbre_six

```

- 1. `randint(1,6)` renvoie un entier tiré au hasard entre 1 et 6 (bornes comprises) : c'est la simulation d'un lancer de dé.
- 2. `if randint(1,6)==6` teste si le lancer du dé donne un 6.
- 3. `de(1000)` simule 1 000 lancers et compte les 6. On ne trouve pas toujours le même résultat : chaque lancer est aléatoire. En moyenne, environ un lancer sur six donne un 6, donc le résultat est proche de $\frac{1000}{6} \approx 167$, mais il varie d'une exécution à l'autre.
- 4. La fonction modélise le fait de lancer *n* fois un dé et renvoie le nombre de 6 obtenus.
- 5. La fréquence d'obtention du 6 est le quotient $\frac{\text{nombre de 6}}{n}$. Seule la dernière ligne change :

```

1     return nbre_six / n

```

Pour *n* grand, cette fréquence se rapproche de $\frac{1}{6} \approx 0,167$.

Exercice 5

Une voiture (valeur 35 000 €) perd 10 % de sa valeur chaque année.

- 1. Perdre 10 % revient à retirer $\frac{10}{100}$ de la valeur chaque année (ce qui équivaut à multiplier par 0,9). On complète :

```
1 def prix(n):
2     prix = 35000
3     for i in range(n):
4         prix = prix - prix * 0.10
5     return prix
```

Au bout de n années, $\text{prix} = 35\,000 \times 0,9^n$.

- 2. Au bout de 10 ans : $35\,000 \times 0,9^{10} \approx 12\,203,74$ €.

- 3. Au bout de 15 ans : $35\,000 \times 0,9^{15} \approx 7\,206,19$ €.

► **Remarque.** La variable `prix` porte le même nom que la fonction `prix` : ce n'est pas une erreur ici, mais pour plus de clarté on préférerait nommer la variable `valeur`.

Exercice 6

Pyramide d'oranges : étage 1 $\rightarrow$ 1, étage 2 $\rightarrow$ 4, étage 3 $\rightarrow$ 9, ...

- 1. À l'étage i , les oranges forment un carré de i oranges de côté : il y en a donc i^2 (étage 1 : $1^2 = 1$; étage 2 : $2^2 = 4$; étage 3 : $3^2 = 9$).

- 2. Le nombre total d'oranges est $1^2 + 2^2 + \dots + n^2$. On parcourt les étages de 1 à n et on ajoute i^2 à chaque tour :

```
1 def pyramide(n):
2     nbre_oranges = 0
3     for i in range(1, n+1):
4         nbre_oranges = nbre_oranges + i**2
5     return nbre_oranges
```

- 3. Pour 10 étages : $1^2 + 2^2 + \dots + 10^2 = 385$. La fonction `pyramide(10)` renvoie **385 oranges**.

- **Pour aller plus loin.** On retrouve la formule $1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$: pour $n = 10$, $\frac{10 \times 11 \times 21}{6} = 385$.

Exercice 7

Fonction `nbre_a` qui compte les lettres « a » d'une phrase.

► **Ce que fait la fonction.** Elle parcourt la phrase lettre par lettre et compte combien valent exactement "a" (le « a » minuscule, sans accent).

► **2. `nbre_a("J'aime les maths")`.** Les « a » sont dans « J'aime » et « maths » : on en compte **2**. La console affiche :

```
>>> nbre_a("J'aime les maths")
2
```

► **3. `nbre_a("Je vais à la plage")`.** On trouve un « a » dans « vais », « la » et « plage », soit **3**.

► **Le piège du « à ».** Le caractère accentué « à » (de « à ») n'est *pas* égal à "a" pour Python : il n'est donc **pas compté**. On obtient bien 3 (et non 4). C'est tout l'intérêt de cette phrase : un caractère accentué est différent de son équivalent non accentué.

Exercice 8 : Boucle For

Fonction `som` ; trace pour $n = 5$, puis modification pour la somme des n premiers entiers impairs.

```
1 def som(n):
2     s=0
3     for k in range(1,n+1):
4         s=s+k
5     return(s)
```

► **1. Trace pour $n = 5$** (`range(1,6)` donne $k = 1, 2, 3, 4, 5$).

k	(départ)	1	2	3	4	5
s	0	1	3	6	10	15

► **2.** Cet algorithme calcule la **somme des entiers de 1 à n** : $1 + 2 + \dots + n$ (pour $n = 5$, on obtient 15).

► **3. Somme des n premiers entiers impairs.** En Seconde, un entier impair s'écrit $2k + 1$ (avec k entier à partir de 0). Avec `range(n)`, k parcourt $0, 1, \dots, n - 1$, donc $2k + 1$ engendre $1, 3, 5, \dots, 2n - 1$: ce sont exactement les n premiers impairs.

```
1 def som(n):
2     s=0
3     for k in range(n):
4         s=s+2*k+1
5     return(s)
```

► Pour $n = 5$: $1 + 3 + 5 + 7 + 9 = 25 = 5^2$. Plus généralement, $1 + 3 + 5 + \dots + (2n - 1) = n^2$.

Exercice 9

Programme *nombre* :

```
1 from math import *
2 def nombre():
3     nombre=0
4     for k in range(1,41):
5         if k%2==0 and k%4!=0:
6             nombre=nombre+1
7         print(k)
8     return nombre
```

► 1. La ligne 5 : `if k%2==0 and k%4!=0`. Elle teste **deux conditions à la fois** (reliées par `and`) :

- `k%2==0` : k est **pair** (divisible par 2) ;
- `k%4!=0` : k n'est **pas** un multiple de 4.

On sélectionne donc les nombres pairs qui ne sont pas des multiples de 4, c'est-à-dire de la forme $4p + 2$.

► 2. Pour k de 1 à 40, l'algorithme **affiche** (avec `print`) tous ces nombres, puis **renvoie leur effectif**. Les nombres affichés sont :

2 6 10 14 18 22 26 30 34 38

Il y en a **10** : la fonction renvoie donc la valeur **10**.

► **Remarque.** L'instruction `from math import *` n'est ici d'aucune utilité : aucune fonction du module `math` n'est utilisée.